M01 – AuthKit: Umstellung von HS256 auf RS256 mit JWKS-Endpoint

Feld	Wert
Arbeitszeitraum	Di 2026-09-29 – Do 2026-10-01, an allen drei Tagen in kleinen Etappen
Commit-Zeitraum	Do 2026-10-01, 22:40 – 23:17
Dokumentiert am	2026-10-04 21:01 CEST
Repository / Service	`authkit` (github.com/Umex10/authkit), Backend `apps/backend` (künftiger `auth-service` von Overex)
Commits	2 (`715af9b` .. `23bd50b`), Branch `feature/rs256-jwks` , Start = Tag `v1-hs256`
Zeitaufwand	ca. 4 h
Bezug	Projektarbeit (5. Semester, Abgabe Jänner 2027) und Grundlage für die Bachelorarbeit (6. Semester)

Ich habe erst am Donnerstag committet, weil das Feature vorher nicht fertig war. Ich wollte keine halben, fehlerhaften Zwischenstände committen.

Ziel

Aus dem Backend von AuthKit soll der `auth-service` von Overex entstehen. Overex besteht aus mehreren Microservices, und jeder davon muss prüfen können, ob ein Access-Token echt ist. Mit dem bisherigen Aufbau ging das nicht sauber.

Ziel war deshalb:

- Nur der `auth-service` kann Tokens ausstellen (Signatur mit einem **privaten** Key, RS256).
- Alle anderen Services prüfen Tokens mit dem **öffentlichen** Key und können selbst keine fälschen.
- Der öffentliche Key ist über einen Endpoint abrufbar, damit andere Services ihn automatisch laden.
- Beim Prüfen eines Tokens gibt es keinen Datenbankzugriff mehr.
- AuthKit bleibt eine klonbare Vorlage, ohne Modus-Schalter.

Ausgangslage

- **HS256 mit einem geteilten Geheimnis:** Ein einziger Key (`jwt.secret`) hat Tokens signiert und geprüft. Der Key stand mit einem Dev-Default in der Konfiguration.
- **Eigener Filter mit Datenbankzugriff:** Der `JwtAuthFilter` hat bei jeder Anfrage den User über die E-Mail im Token aus der Datenbank geladen und daraus die Rolle gelesen.
- **Token-Inhalt:** `sub` war die E-Mail, Rollen standen nicht im Token, und der Token-Typ (`ACCESS` / `REFRESH`) wurde nirgends geprüft.

Für Microservices passt das aus zwei Gründen nicht:

1. **Wer prüfen kann, kann auch fälschen.** Jeder Service hätte den geheimen Key gebraucht. Damit hätte jeder Service auch gültige Tokens erstellen können.
2. **Jeder Service hat seine eigene Datenbank.** Andere Services können die User des `auth-service` nicht laden. Der Filter hätte dort nicht funktioniert.

Umsetzung (chronologisch)

1. **Di–Mi** – Bestehenden Code durchgearbeitet. Tag `v1-hs256` auf den HS256-Stand gesetzt und einen eigenen Branch angelegt. Einen ersten, komplett von Claude erstellten Umbau habe ich verworfen, weil ich jeden Schritt selbst bauen und verstehen wollte.
2. **Mi** – RSA-Schlüsselpaar mit openssl erzeugt. Die Keys liegen lokal und kommen weder ins Repo noch ins Docker-Image.
3. **Do** – Keys in Spring geladen (`KeyConfig`) und `JwtService` auf RS256 umgestellt. Tokens enthalten jetzt die User-ID als `sub` , Access-Tokens zusätzlich `email` und `roles` .

4. **Do** – Den eigenen Filter durch den OAuth2 Resource Server von Spring Security ersetzt. Er prüft Access-Tokens nur mit dem öffentlichen Key (`87bec2b`).
5. **Do** – JWKS-Endpoint gebaut, über den andere Services den öffentlichen Key abrufen (`23bd50b`).

Geänderte Dateien

Datei	Art	Zweck
`auth/config/KeyConfig.java`	neu	lädt privaten und öffentlichen Key aus PEM-Dateien
`auth/JwtService.java`	geändert	signiert mit RS256, neue Claims, prüft Refresh-Tokens mit dem öffentlichen Key
`auth/security/config/SecurityConfig.java`	geändert	Resource Server statt eigenem Filter, Prüfung des Token-Typs, Rollen aus dem Token
`auth/security/JwtAuthFilter.java`	gelöscht	ersetzt durch den Resource Server
`auth/JwksController.java`	neu	stellt den öffentlichen Key unter `/.well-known/jwks.json` bereit
`user/MeController.java`	geändert	liest die User-ID direkt aus dem geprüften Token

Pfade relativ zu `apps/backend/src/main/java/com/authkit/backend/` . Dazu kleinere Anpassungen in `AuthService` , `AuthController` , `pom.xml` , Konfiguration und Tests.

Entscheidungen

Entscheidung	Verworfene Alternativen	Begründung
RS256 statt HS256	HS256 mit demselben Secret in allen Services	Bei RS256 prüfen Services mit dem öffentlichen Key und können keine Tokens ausstellen.
Spring Security Resource Server statt eigenem Filter	eigenen Filter auf RS256 umbauen	Gepflegter Standardcode, prüft ohne Datenbank. Die anderen Overex-Services nutzen später dieselbe Abhängigkeit.
Keys als Dateien, Pfad per Konfiguration überschreibbar	Default-Key im Repo; Key im Image	Der private Key darf nie ins Repo oder Image. Ohne Key startet die App nicht, statt unsicher zu laufen.
`sub` = User-ID, Rollen im Access-Token	E-Mail als `sub` ; Rollen bei jeder Anfrage aus der Datenbank	Die ID ändert sich nie. Ohne Datenbank muss die Rolle im Token stehen, damit `hasRole('ADMIN')` funktioniert.
Refresh-Token enthält nur die User-ID, der User wird beim Refresh frisch geladen	alle Claims auch im Refresh-Token	So sind E-Mail und Rolle im neuen Access-Token immer aktuell.
Token-Typ wird geprüft: nur `ACCESS` im Header, nur `REFRESH` beim Refresh	keine Prüfung (wie vorher)	Sonst wäre ein 30 Tage gültiger Refresh-Token als Access-Token nutzbar.
Andere Services prüfen lokal mit dem öffentlichen Key (JWKS)	bei jeder Anfrage beim `auth-service` nachfragen	Nachfragen würde den `auth-service` bei vielen gleichzeitigen Käufen zum Flaschenhals machen.
Kompromiss: Ein gesperrter User bleibt bis zum Ablauf des Access-Tokens gültig (max. 15 Minuten)	bei jeder Anfrage die Datenbank fragen	Bewusst in Kauf genommen für die Prüfung ohne Datenbank. Die kurze Lebensdauer begrenzt das Risiko.

Neu gelernt

- **HS256 vs. RS256**: Bei HS256 signiert und prüft derselbe Key. Bei RS256 signiert der private Key, der öffentliche prüft. Aus dem öffentlichen lässt sich der private nicht berechnen.
- **Wie eine Signatur geprüft wird**: Der Prüfer berechnet den Hash von Header und Payload neu und vergleicht ihn mit der Signatur. Wird die Payload verändert, passt die Signatur nicht mehr.

- **PEM und Base64**: Ein Key in einer PEM-Datei ist Base64-kodierter Binärinhalt zwischen `BEGIN` / `END` -Zeilen. Base64 ist eine Kodierung, keine Verschlüsselung.
- **Spring Security Resource Server**: Der `JwtDecoder` prüft das Token, der `JwtAuthenticationConverter` macht aus dem Claim `roles` die Berechtigungen `ROLE_...` , mit denen `@PreAuthorize` arbeitet.
- **JWK und JWKS**: Ein JWK ist ein Key als JSON, ein JWKS eine Liste davon. Andere Services laden es über eine URL und speichern es im Arbeitsspeicher zwischen. Mit einer `kid` im Token findet der Prüfer den passenden Key, was später Key-Rotation ermöglicht.
- **Docker-Image vs. Container**: Geheimnisse gehören nicht ins Image, das verteilt wird. Sie werden zur Laufzeit per Volume in den Container gegeben.
- **Checked vs. unchecked Exceptions**: Der Unterschied betrifft nur den Compiler. Unchecked Exceptions können bis zu einem zentralen Handler wandern, der daraus eine HTTP-Antwort macht.

Tests & Verifikation

Test	Ergebnis
Sign-up und Sign-in im Browser nach Neustart des Backends	funktioniert
`./mvnw test` (Stand `87bec2b`)	20 von 24 Tests grün. `MeControllerTest` (4 Tests) scheitert, weil im Testkontext der öffentliche Key fehlt.
`./mvnw clean compile` (Stand `23bd50b`)	erfolgreich
Commits auf Key-Material geprüft	kein Key im Repo
Aufruf des JWKS-Endpoints	noch nicht getestet

Probleme & Lösungen

- **Problem:** Nach der Umstellung von `sub` auf die User-ID hat der Refresh den User noch per E-Mail gesucht. → **Lösung:** User per ID laden.
- **Problem:** `loadUserByUsername` wurde auf die User-ID umgestellt, Spring ruft es beim Login aber mit der E-Mail auf. → **Lösung:** Login bleibt bei der E-Mail, nur der Refresh lädt per ID.
- **Problem:** Die Prüfung auf `REFRESH` war umgekehrt und verglich einen String mit einem Enum. → **Lösung:** korrekter Vergleich über `name()` , Fehler als 401 statt 500.
- **Problem:** Der Backend-Container lief noch mit altem Code, und die Keys sind absichtlich nicht im Image. → **Lösung:** Backend vorerst lokal mit Maven gestartet. `docker-compose` passe ich noch an.

Eingesetzte Hilfsmittel

Ich habe Claude (Claude Code) als Lernbegleiter eingesetzt. Claude hat mir die Konzepte erklärt und die Schritte vorgegeben. Den Großteil des Umbaus habe ich selbst geschrieben. Auf meinen Wunsch hat Claude einzelne Teile übernommen: Teile von `KeyConfig` , den `JwtAuthenticationConverter` , den `JwksController` , das Aufräumen von Javadoc und Imports sowie das Anpassen der Tests, damit sie kompilieren. Einen ersten kompletten Umbau durch Claude habe ich verworfen.

Hinweis: Die FH-Richtlinien zur Kennzeichnung von KI-Unterstützung sind zu beachten.

Offene Punkte / Nächste Schritte

- **Tests überarbeiten:** `MeControllerTest` und den vollständigen Kontexttest lauffähig machen, ohne lokale Keys. Tests für falsche Signatur, abgelaufenes Token, Refresh-Token als Access-Token, ADMIN-Rolle und den JWKS-Endpoint ergänzen.
- **Beispiel-Microservice bauen:** einen kleinen separaten Service anlegen, der AuthKit nutzt und Tokens über den JWKS-Endpoint prüft. Damit teste ich das Zusammenspiel vollständig.
- Danach: `docker-compose` (Keys per Volume), README (Schlüsselerzeugung, 15-Minuten-Kompromiss), optional `kid` für Key-Rotation.

Commit-Verlauf

Datum	Hash	Nachricht

2026-10-01 22:40	`87bec2b`	feat(auth): sign JWTs with RS256 and verify them via the OAuth2 resource server
2026-10-01 23:17	`23bd50b`	feat(auth): publish the public signing key as a JWKS